

Integración de sistemas

Service-Oriented Architecture (SOA)

Damián Nicolalde R.

2021

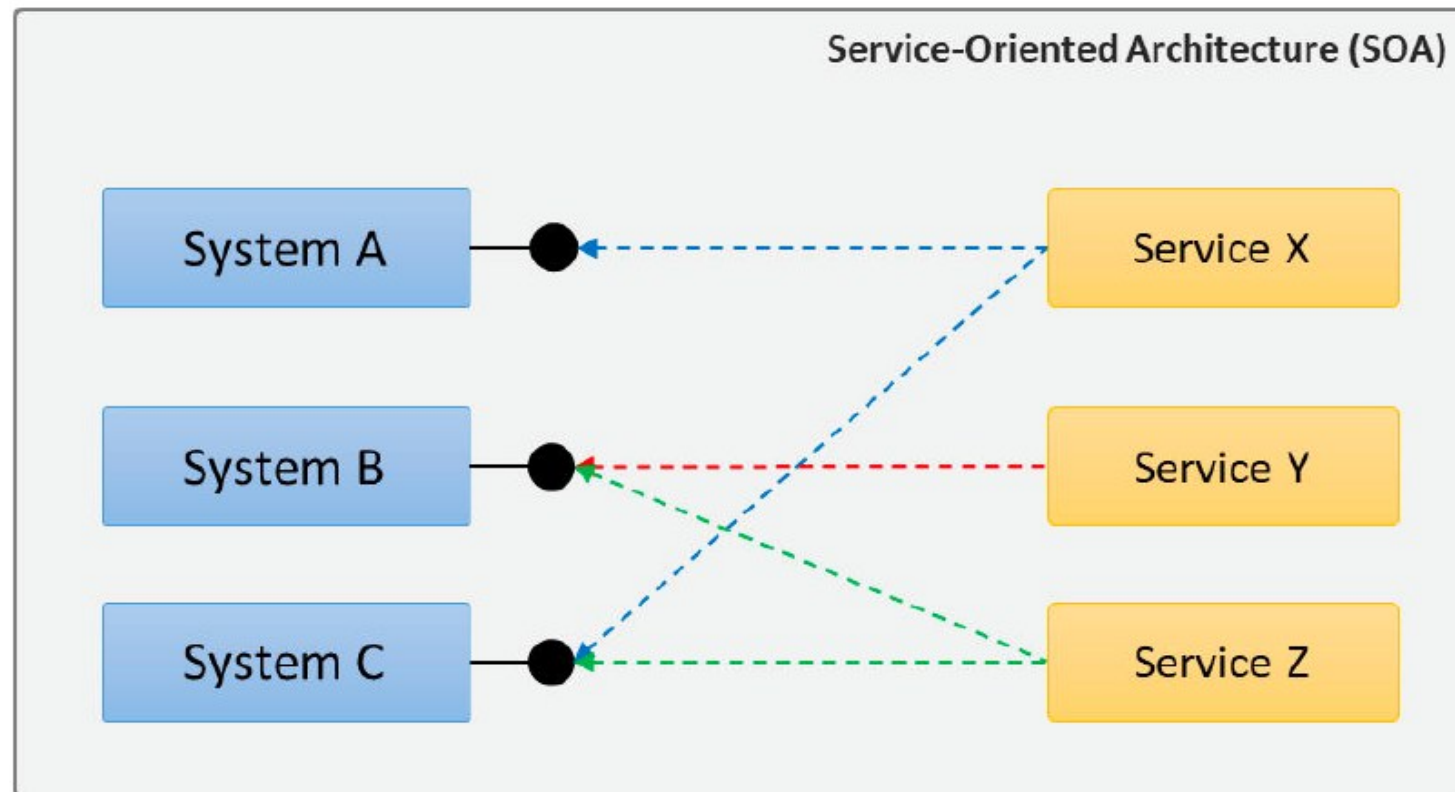
danicolalde@puce.edu.ec

Service-Oriented Architecture (SOA)

- La arquitectura SOA (Service-Oriented Architecture), incentiva a que las empresas creen servicios interoperables que puedan ser aprovechados por toda la organización. Dichos servicios deben de ser contruidos utilizando estándares abiertos, lo que permita que pueden ser consumidos por cualquier aplicación sin importar en que tecnología esté desarrollada.
- SOA solo promueva la creación de servicios para lograr la integración de aplicaciones, por lo que se podría decir que CORBA es el precursor de SOA, aunque al momento de crear CORBA no existía el término de SOA. Por su parte, el nacimiento de SOA como arquitectura no solo trae el concepto de crear servicios si no un nuevo paradigma que revoluciono la forma en que construimos software de hoy en día.

Como se estructura una arquitectura de SOA

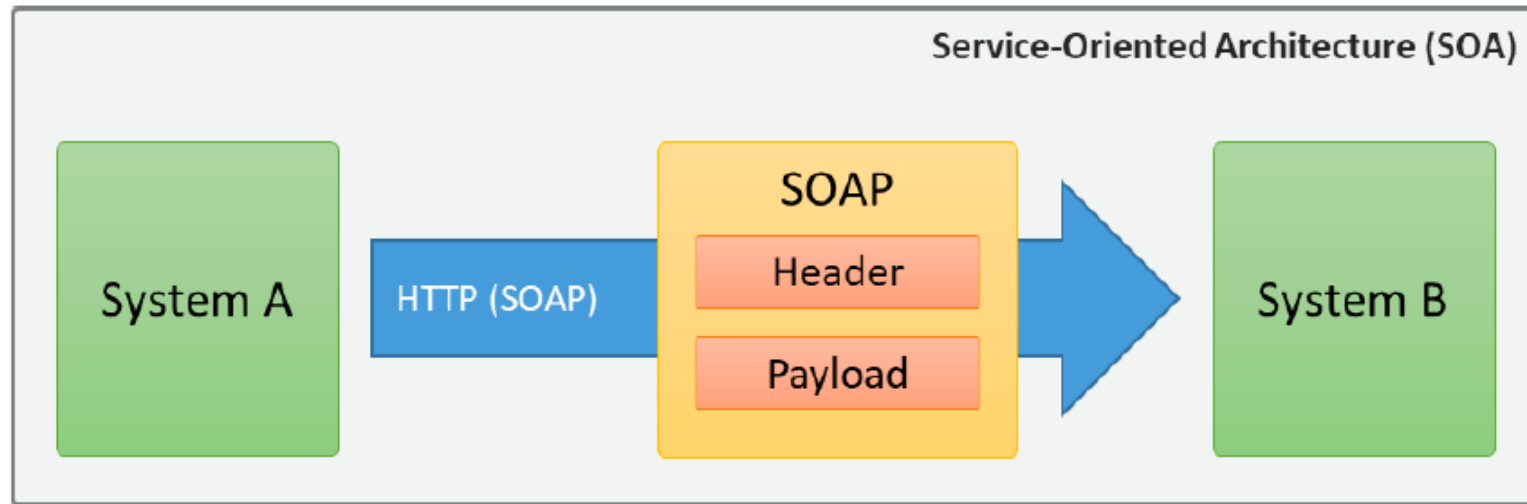
- Lo primero que tenemos que entender al momento de implementar una arquitectura SOA es que, todas las aplicaciones que construimos están diseñadas para ser integradas con otras aplicaciones, por lo que deben de exponer como servicios todas las operaciones necesarias para que otras aplicaciones puedan interactuar con ella.



Independiente de la tecnología

- Lo primero que debemos de analizar es que, los servicios deben ser contruidos con estándares abiertos, lo que quiere decir que es independiente del sistema operativo o del lenguaje de programación. Hoy en día conocemos a estos servicios como Web Services.
- Para que un servicio sea considerado un Web Service, debe de publicar un WSDL (Web Services Description Language), el cual es un contrato que describe las operaciones que expone el servicio, los tipos de datos esperados en formato XSD (XML Schema).
- Este contrato (WSDL) será utilizado por el consumidor para crear mensajes en formato XML para enviar la información, los cuales deberán de cumplir al pie de la letra la estructura definida en el contrato.
- Finalmente, el mensaje es envuelto en el protocolo SOAP (Simple Object Access Protocol), el cual es otro protocolo estándar que divide el mensaje en dos secciones, el header y el payload, el primero son una serie de parámetros de cabecera de control, que ayudan a los sistemas a entender el payload, por otra parte, el payload es el mensaje transmitido como tal (XML).

Independiente de la tecnología



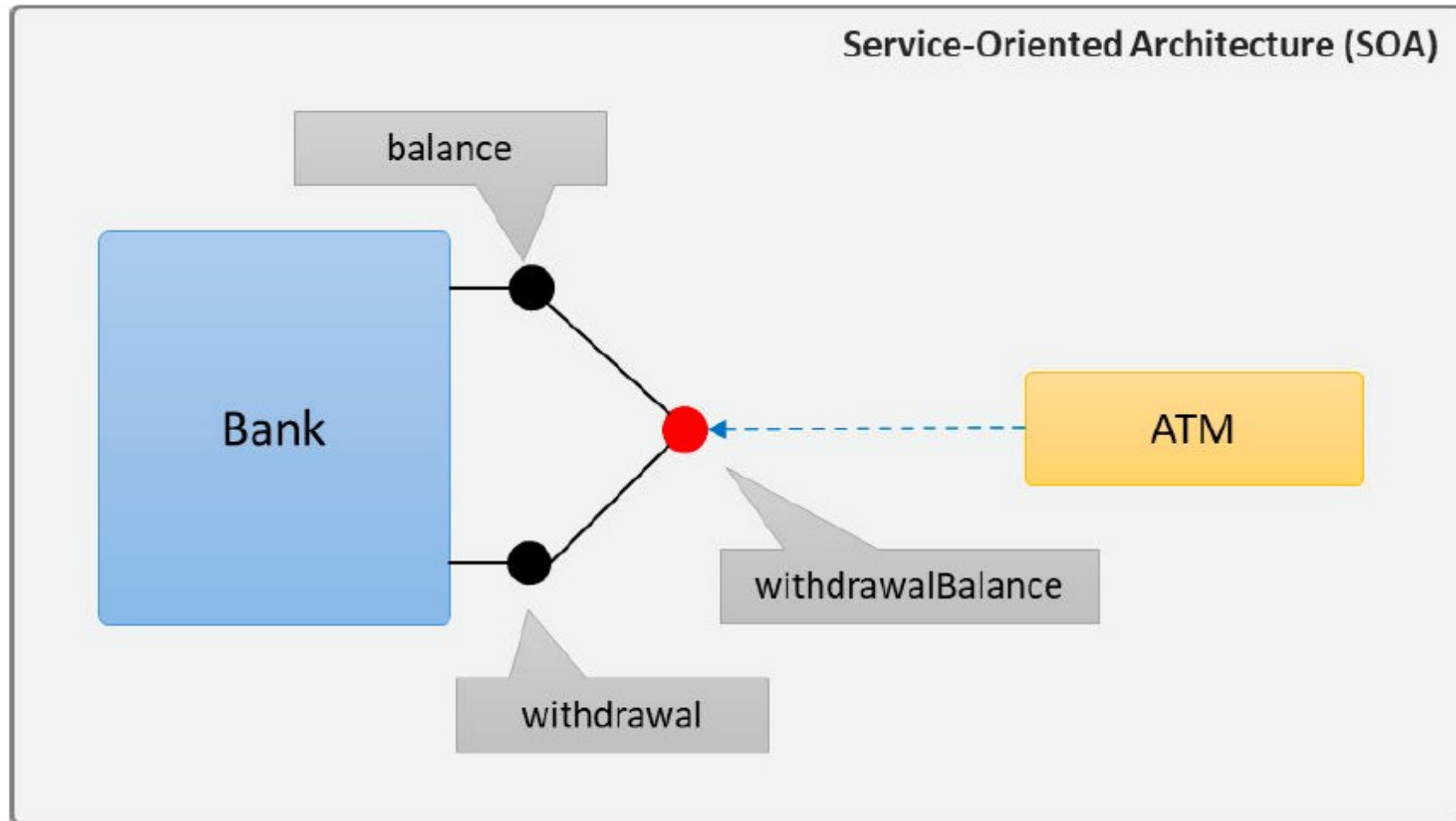
Reutilización

- Cuando nació DCOM y CORBA no existía una cultura de reutilización, por lo que lo que se exponía eran por lo general operaciones de negocio complejas que realizaban varios pasos, lo que hacía muy difícil que una operación se pudiera ser reutilizada por otras aplicaciones, en este sentido, una operación era construida específicamente para una integración específica.
- Por el contrario, SOA promueve la construcción de servicios que hagan una sola tarea, lo que hace que en lugar de tener 1 servicio que haga 10 cosas, mejor hacemos 10 servicios que hagan 1 cosas. En principio se puede escuchar raro tener que crear más servicios, sin embargo, al construir servicios que hagan una sola tarea los hace mucho más reutilizables, por otra parte, es posible crear nuevos servicios basado en los anteriores, lo que conocemos como servicios compuestos.

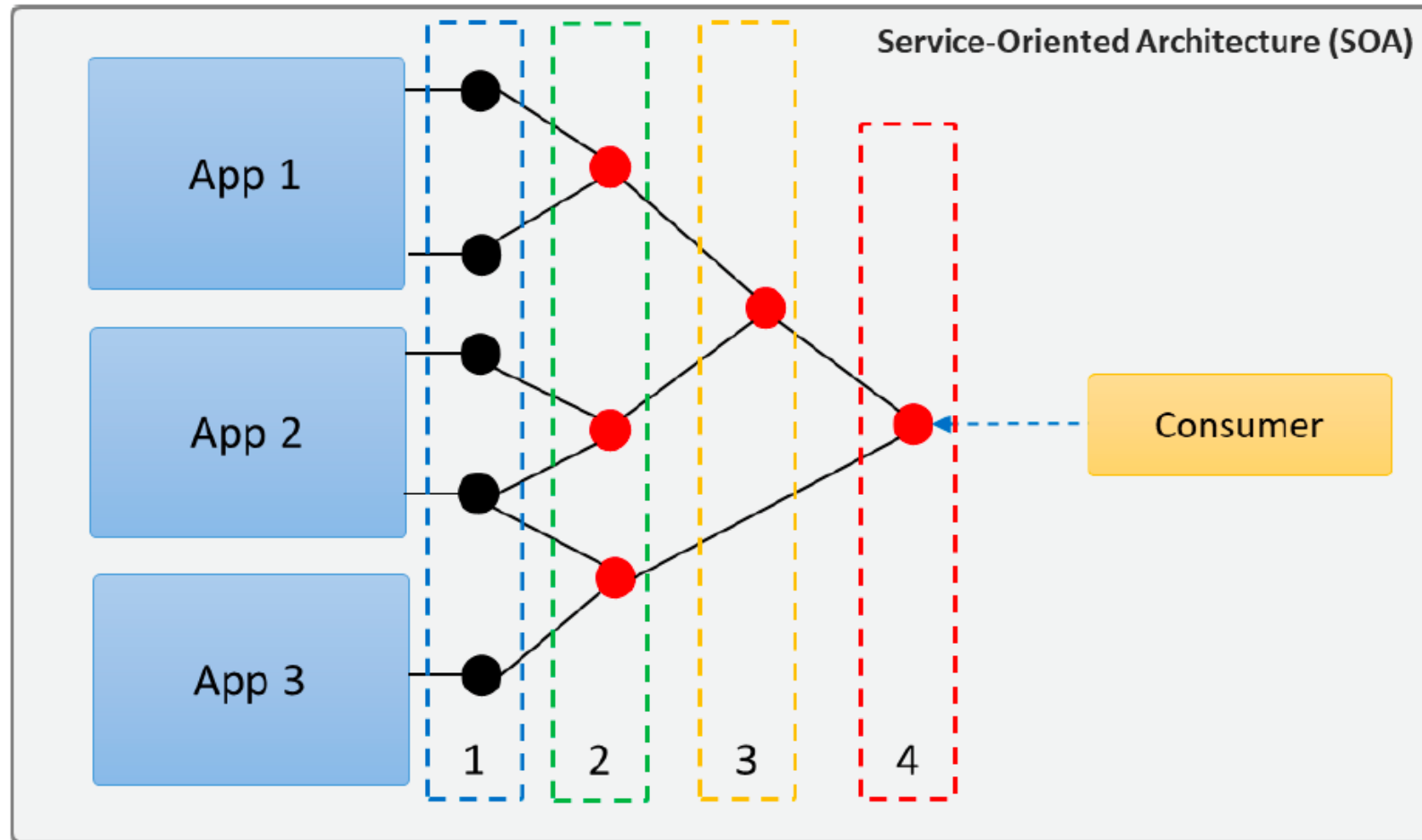
Reutilización

- Imagina el siguiente escenario. Eres el programador de un Banco y te han pedido que realices una serie de servicios para operar un cajero automático, en el cual puedes consultar el saldo y realizar retiros. Como regla, realizar un retiro concluye con mostrarle al usuario su nuevo saldo. La respuesta obvia sería crear dos servicios, uno que consulte el saldo y otro que haga el retiro y luego consulte el saldo, sin embargo, para que consultamos nuevamente el saldo en el retiro si ya tenemos un servicio que lo hace, entonces mejor realizamos el retiro y luego llamamos al servicio de consulta de saldo:

Reutilización



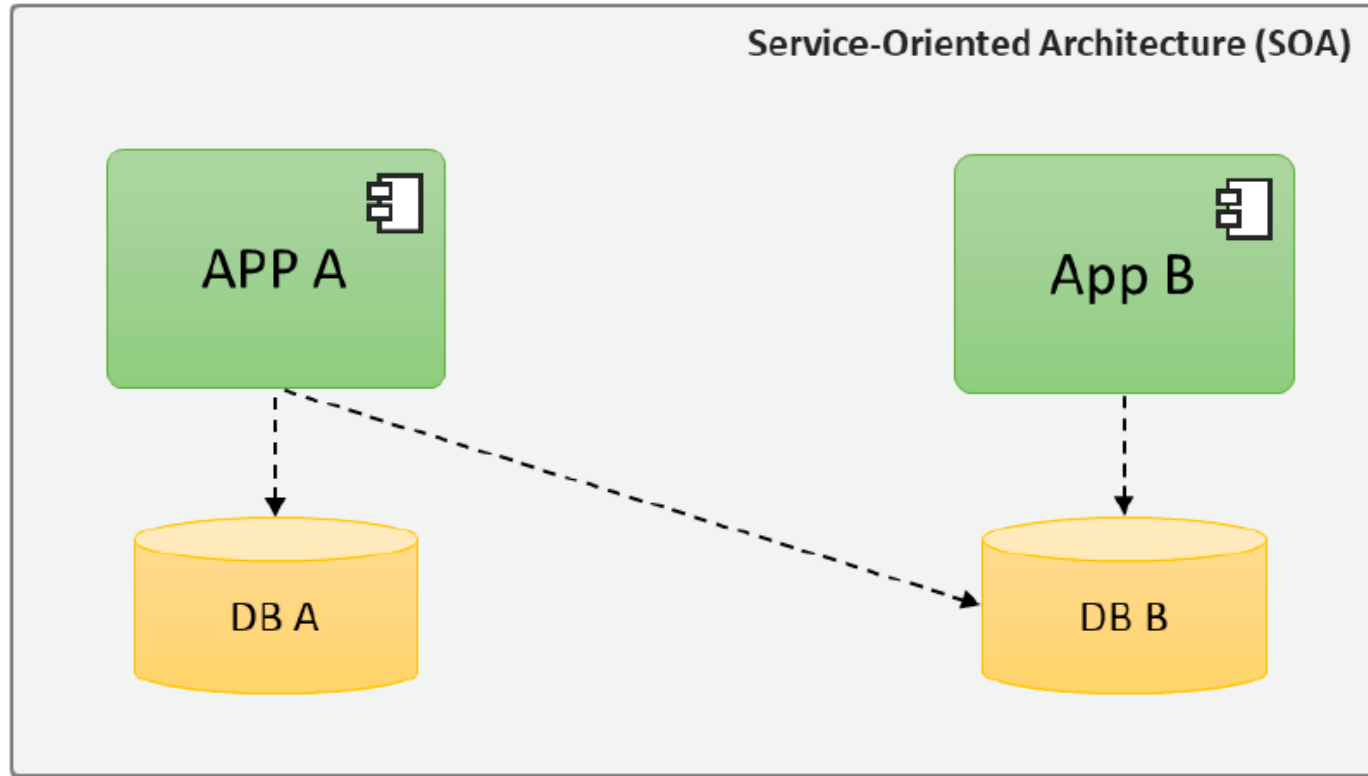
Reutilización



Encapsulamiento

- La encapsulación es una de las características más importantes de SOA, pues permite ocultar la complejidad de un sistema detrás de una serie de servicios de alto nivel. Solo imagina, que es más fácil, pedirle a un servicio que te consulte un pedido con solo el número de pedido, o tener que ir a la base de dato del sistema, saber todas las tablas y campos que tenemos que consultar y retornarlas. Creo que no queda duda que el servicio es mucho más simple, pues no nos tenemos que preocupar en donde están los datos ni como los tenemos que retornar, pues le servicio lo hace por ti.

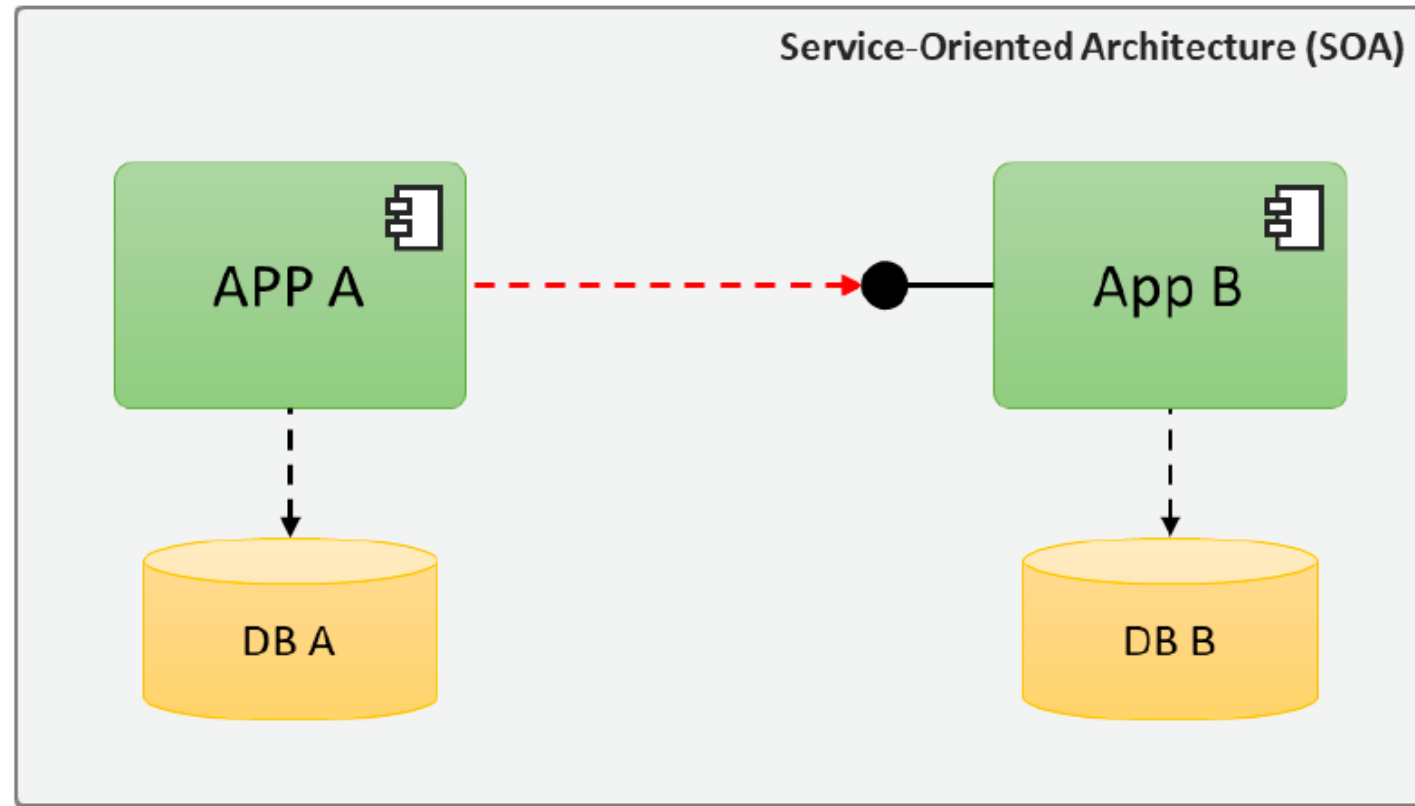
Encapsulamiento



No compartas las bases de datos

Hoy en día es muy mal visto que una aplicación ajena modifique directamente la base de datos, pues puede llevar a problemas serios de integridad de los datos.

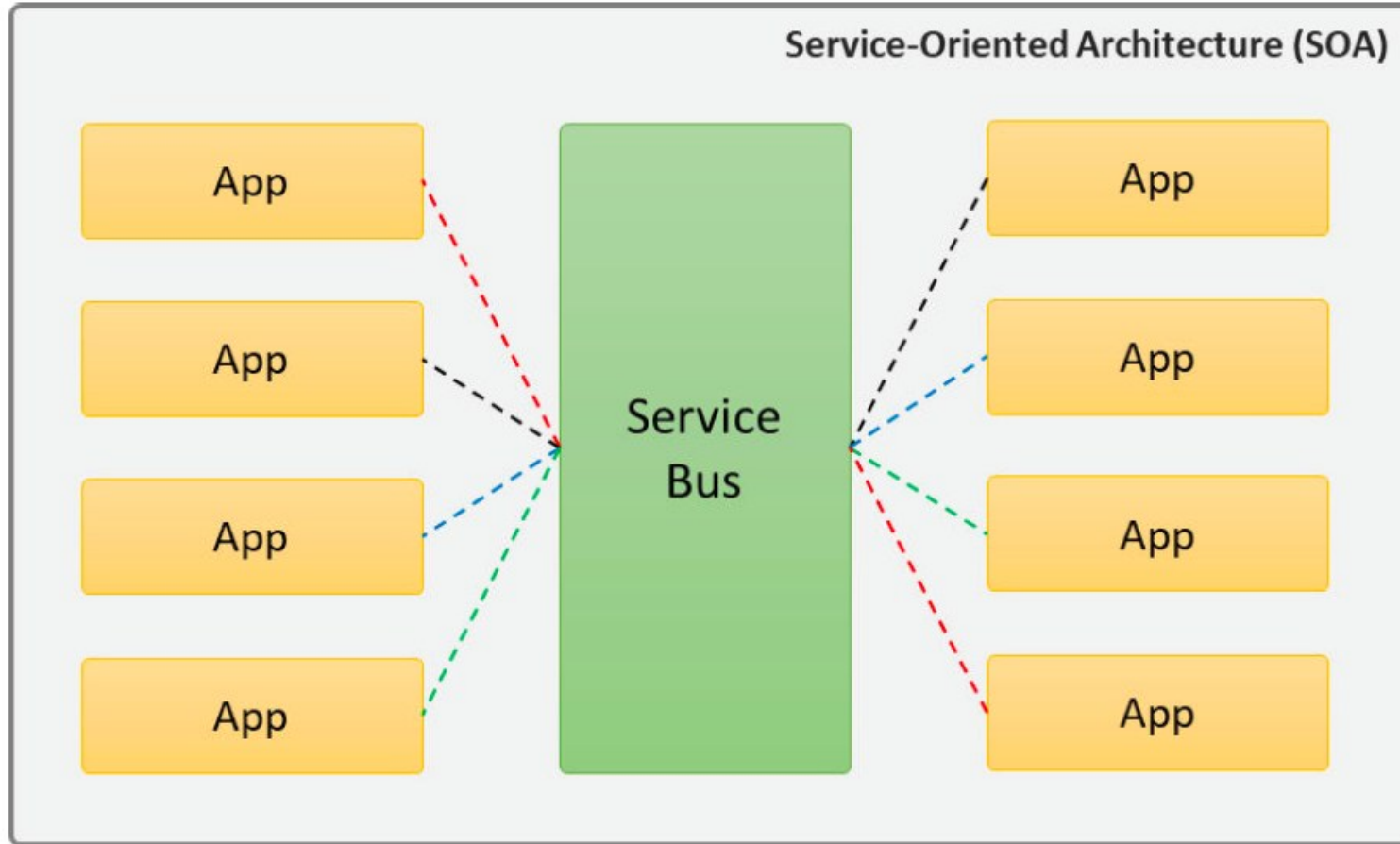
Encapsulamiento



Enterprise Service Bus (ESB)

- Uno de los componentes más importantes cuando hablamos de SOA son los Services Bus, el cual es un componente que permite crear abstracciones de los servicios expuestos de una aplicación para evitar el acoplamiento entre los dos sistemas.
- Una de las principales problemáticas cuando comenzamos a trabajar con servicios es que, se comienza a crear una serie de conexiones de punto a punto entre las aplicaciones, lo que hace difícil entender que aplicaciones se comunica con cual, además de que se crea un fuerte acoplamiento entre las aplicaciones, llegando a los extremos que una aplicación no puede ser desplegada si los servicios requeridos no están disponibles, lo que ocasiona una falla en cascada de las aplicaciones. Para solventar este problema, El Service Bus permite crear abstracciones de los servicios expuestos por las aplicaciones, lo que hace que las aplicaciones se comuniquen con el Service Bus, el cual, a su vez, se comunicará con la aplicación final.

Enterprise Service Bus (ESB)



Características de SOA

- Las aplicaciones son pensadas desde el inicio para ser integradas con otras, por lo que exponen una serie de servicios para exponer su funcionalidad al resto de aplicaciones.
- Existe un grado muy alto de desacoplamiento entre las aplicaciones, debido a que trabajan con contratos y no referencias directas a las otras aplicaciones.
- Las aplicaciones se comunican mediante un Enterprise Service Bus (No es la regla, pero es lo más común).
- La comunicación se puede dar por diferentes protocolos (aunque HTTP es el más utilizado).
- Los mensajes intercambiados entre las diferentes aplicaciones siguen un formato específico el cual está definido en el contrato del servicio (WSDL).
- Los servicios son distribuidos, lo que quiere decir que los servicios pueden estar en cualquier parte de la red, ya sea local o remota y en cualquier parte geográfica.

Ventajas

- **Reduce el acoplamiento:** La comunicación basada en contratos permite desacoplar las aplicaciones, ya que no existe referencias a la aplicación, si no a un servicio que puede estar en cualquier parte.
- **Testabilidad:** Las aplicaciones basadas en SOA son muy fáciles de probar, pues es posible probar de forma individual cada servicio.
- **Reutilización:** Los servicios SOA son desarrollados para hacer una sola tarea, por lo que facilita que otras aplicaciones reutilicen los servicios que existen.
- **Agilidad:** Permite crear rápidamente nuevos servicios a partir de otros existentes.
- **Escalabilidad:** Los servicios SOA son muy fáciles de escalar, sobre todo porque permite agregar varias instancias de un servicio y balancear la carga desde el ESB.
- **Interoperable:** La interoperabilidad es uno de los factores por lo que existe esta arquitectura y es que permite que aplicaciones heterogéneas se comuniquen de forma homogénea mediante el uso de estándares abiertos.

Desventajas

- **Performance:** La arquitectura SOA no se caracteriza por el performance, pues la comunicación por la red y la distribución de los servicios agregan una latencia significativa en la comunicación.
- **Volumetría:** SOA no se lleva con grandes volúmenes por petición, por lo que para procesar mucha información es recomendable utilizar otras tecnologías.
- **Gobierno:** Para que SOA pueda ser implementado correctamente, es necesario crear un Gobierno que ponga orden sobre el uso y la creación de nuevos servicios, lo que puede representar una carga adicional a la empresa.
- **Más puntos de falla:** Debido a que SOA es una arquitectura distribuida, es necesario implementar estrategias de falla, pues se incrementa el número de puntos de falla.

Cuando no usar SOA

1. Para aplicaciones que tiene que procesar o enviar masivas cantidades de datos. Es importante diferenciar entre grandes volúmenes de datos y grandes volúmenes de peticiones, con SOA podemos procesar grandes volúmenes de mensajes, pues podemos clusterizar los servicios para atender muchas peticiones, pero es diferente cuando estas peticiones tienen en solitud o respuesta una masiva cantidad de datos, este último es un inconveniente para SOA pues puede poner en jaque a la arquitectura.
2. Cuando el performance es lo más importante en nuestra aplicación, SOA no será una buena idea, pues la arquitectura SOA agrega muchas capas que hace que el procesamiento de los datos no sea tan rápido.
3. Implementar SOA puede resultar atractivo, sin embargo, hay que tomar en cuenta que su implementación requiere de administración y gobierno, por lo que para empresas pequeñas o que no tiene procesos maduros puede llegar a ser un obstáculo en lugar de una ventaja. Si no te sientes listo para asumir estas responsabilidades, será mejor esperar un poco.

Conclusiones

- Como hemos podido analizar en este tema, SOA es una arquitectura muy flexible que permite la interoperabilidad entre las diferentes aplicaciones empresariales, ya que utiliza una serie de estándares abiertos que permite que sean independientes de la tecnología o la plataforma, sin embargo, SOA es uno de los estilos arquitectónicos más complejos por la cantidad de componentes que interviene, el gobierno y toda una serie de lineamientos que se deben de cumplir para implementarla correctamente.